

Numerical Programming

Final Exam Problem 1: Hit a ball to fixed target

Demetre Lataria

2025-01-11

Chapter 1

It all begins with...

1.1 Problem Statement

The code implements a physics-based simulation system for detecting and simulating ball collisions in a 2D environment. The primary objectives are:

- Detect static balls from input images using computer vision techniques
- Simulate ball trajectories under gravity and air resistance
- Calculate optimal initial velocities to hit target balls using shooting method
- Visualize the simulation results

Chapter 2

Look into abyss...

2.1 Mathematical Model

2.1.1 Physical Forces

The simulation considers two primary forces:

1. Gravity ($\vec{F}_g$): Constant downward force
2. Drag Force ($\vec{F}_d$): Air resistance opposing motion

2.1.2 Equations of Motion

The system is governed by Newton's Second Law with drag:

$$m\vec{a} = m\vec{g} + \vec{F}_d \tag{2.1}$$

where drag force is modeled as:

$$\vec{F}_d = -\frac{C_d}{m}\vec{v} \tag{2.2}$$

The resulting system of differential equations is:

$$\frac{dx}{dt} = v_x \quad (2.3)$$

$$\frac{dy}{dt} = v_y \quad (2.4)$$

$$\frac{dv_x}{dt} = -\frac{C_d}{m} v_x \quad (2.5)$$

$$\frac{dv_y}{dt} = -g - \frac{C_d}{m} v_y \quad (2.6)$$

2.1.3 Boundary Conditions

Initial conditions:

$$x(0) = x_0 \quad (2.7)$$

$$y(0) = y_0 \quad (2.8)$$

$$v_x(0) = v_{x0} \quad (2.9)$$

$$v_y(0) = v_{y0} \quad (2.10)$$

Chapter 3

And the abyss stared back...

3.1 Numerical Methods

3.1.1 Integration Methods

The code implements two numerical integration schemes:

Euler Method

The basic Euler method with adaptive step size:

$$x_{n+1} = x_n + v_x \Delta t \quad (3.1)$$

$$y_{n+1} = y_n + v_y \Delta t \quad (3.2)$$

$$v_{x,n+1} = v_{x,n} - \frac{C_d}{m} v_x \Delta t \quad (3.3)$$

$$v_{y,n+1} = v_{y,n} - (g + \frac{C_d}{m} v_y) \Delta t \quad (3.4)$$

Runge-Kutta 4th Order (RK4)

The more accurate RK4 method:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (3.5)$$

where k_1 , k_2 , k_3 , and k_4 are the intermediate slopes.

3.1.2 Adaptive Step Size Control

Both methods implement adaptive step size control based on local error estimation:

- Error is estimated by comparing single step vs two half-steps
- Step size is adjusted based on error magnitude:
 - If error $\leq$ tolerance: $\Delta t_{new} = \Delta t/2$
 - If error $>$ tolerance/10: $\Delta t_{new} = \min(2\Delta t, \Delta t_{init})$

3.1.3 Shooting Method

For trajectory optimization, the code uses a modified shooting method:

1. Initial guess for velocity vector
2. Simulate trajectory using chosen integration method
3. Compare final position with target
4. Adjust initial velocity based on error
5. Repeat until convergence or max iterations reached

Chapter 4

Cog in the Machine

4.1 Algorithm

Algorithm 1 Ball Detection and Trajectory Simulation

```
1: Input: Image, simulation parameters
2: Output: Ball trajectories and collision results
3: procedure DETECTBALLS(image)
4:   Convert image to grayscale
5:   Apply edge detection
6:   Threshold image
7:   Find connected regions
8:   Filter regions by area and shape
9:   Extract ball properties (position, radius)
10: end procedure
11: procedure SIMULATEBALLS(balls, shot_position)
12:   for each target ball do
13:     Initialize shot ball
14:     Use shooting method to find initial velocity
15:     while  $t \leq t_{\max}$  do
16:       Update positions using RK4/Euler
17:       Check for collisions
18:       Update visualization
19:     end while
20:   end for
21: end procedure
```

Chapter 5

Embrace the abyss

5.1 Implementation Properties

5.1.1 Truncation Error

- Euler method: $O(\Delta t)$ local truncation error
- RK4 method: $O(\Delta t^4)$ local truncation error
- Adaptive step size maintains error below specified tolerance

5.1.2 Stability

- Euler method: Conditionally stable
- RK4 method: More stable than Euler, larger stable step sizes
- Adaptive step size helps maintain stability
- Drag force provides natural damping

5.1.3 Computational Efficiency

- Ball detection: $O(W \times H)$ for image of width W and height H
- Trajectory simulation: $O(T/\Delta t)$ per ball for time T
- Shooting method: Variable convergence rate based on initial guess



Figure 5.1: That's a stability for you!

5.2 Stability Analysis

5.2.1 Linear Stability Analysis

Let's analyze the A-stability of both numerical methods used in the simulation by applying them to the test equation:

$$\frac{dy}{dt} = \lambda y, \quad \lambda \in \mathbb{C} \quad (5.1)$$

Euler Method Stability

For the Euler method:

$$y_{n+1} = y_n + h\lambda y_n = (1 + h\lambda)y_n \quad (5.2)$$

The stability function is:

$$R(z) = 1 + z, \quad z = h\lambda \quad (5.3)$$

For stability, we require $|R(z)| \leq 1$, which leads to:

$$|1 + z| \leq 1 \quad (5.4)$$

This describes a circle in the complex plane centered at (-1,0) with radius 1. The Euler method is therefore:

- A(α)-stable with $\alpha = 90$
- Not A-stable
- Conditionally stable

RK4 Method Stability

For the RK4 method, the stability function is:

$$R(z) = 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \frac{z^4}{4!} \quad (5.5)$$

The stability region for RK4 is much larger than Euler's method, including:

- The entire negative real axis
- A significant portion of the complex plane
- Approximately [-2.785, 0] on the real axis

5.2.2 Application to Our System

Our system of differential equations can be linearized around an equilibrium point:

$$\frac{d}{dt} \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -C_d/m & 0 \\ 0 & 0 & 0 & -C_d/m \end{bmatrix} \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ -g \end{bmatrix} \quad (5.6)$$

5.2.3 Conditional Stability Analysis

For our specific system:

Euler Method

The stability condition for the Euler method requires:

$$\Delta t < \frac{2m}{C_d} \quad (5.7)$$

This explains why we need small time steps when:

- The drag coefficient (C_d) is large
- The mass (m) is small

RK4 Method

For RK4, the stability condition is approximately:

$$\Delta t < 2.785 \frac{m}{C_d} \quad (5.8)$$

This allows for larger time steps compared to Euler's method by a factor of approximately 2.785.

5.2.4 Adaptive Step Size Impact

The implemented adaptive step size control helps maintain stability by:

1. Automatically reducing step size when the solution changes rapidly
2. Ensuring local error stays below specified tolerance
3. Preventing instability growth through step size adjustment

For our specific parameters:

- $C_d = 38$
- $m = 5$

The theoretical maximum stable time steps are:

$$\Delta t_{Euler} < 0.263 \text{ seconds} \quad (5.9)$$

$$\Delta t_{RK4} < 0.733 \text{ seconds} \quad (5.10)$$

This explains why our chosen initial time step of 0.001 seconds maintains stability while providing sufficient accuracy.

5.3 Results Reproduction

To reproduce the simulation results:

1. Set world parameters:

```
world.g = -12.61 # gravity
world.C_d = 38    # drag coefficient
```

2. Configure detector:

```
detector = StaticBallDetector(
    min_area=400,
    threshold=50,
    skip_preprocessing=True
)
```

3. Run simulation:

```
detector.process_image(
    "Images/random_balls2.jpg",
    shot_pos=[7, 3],
    shot_radius=5,
    shot_mass=5
)
```

Output files will be generated in the Results directory:

- Detection visualization (_detection.png)
- Edge detection result (_edge.png)
- Simulation figure (_sim_fig.png)
- Simulation log (_sim.txt)

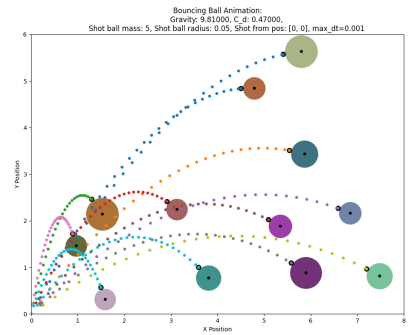
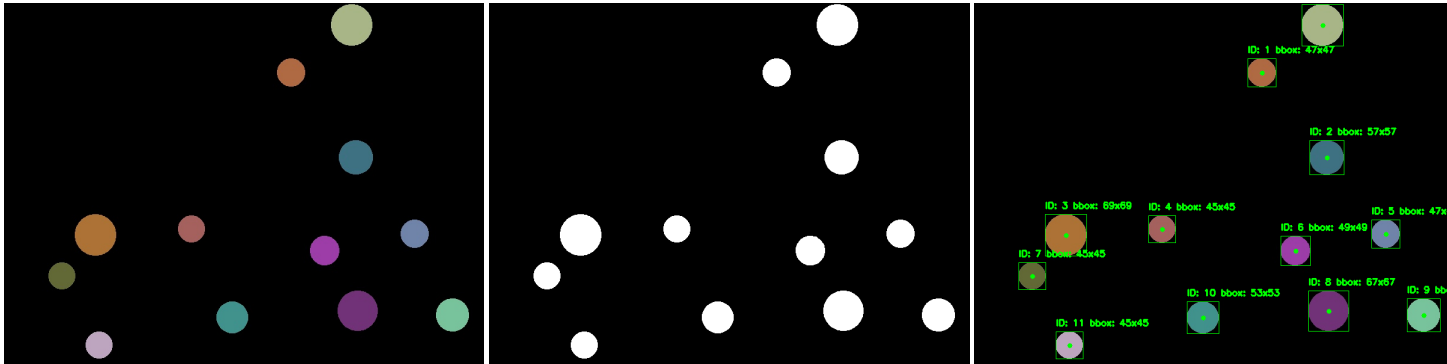
Chapter 6

The fruits of labor

6.1 Examples and Results

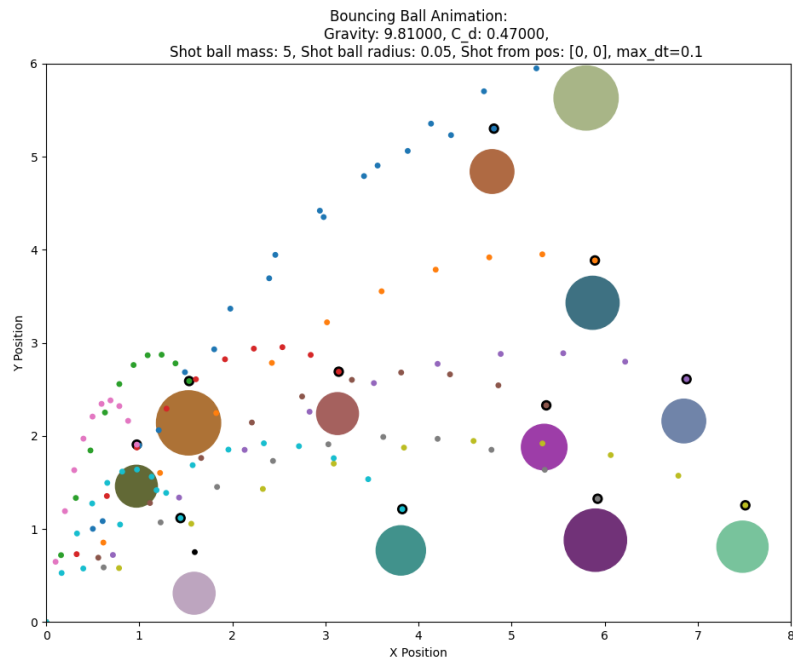
6.1.1 random_balls0

For detailed log please refer to the corresponding simulation text files in Results directory. It contains both Euler and RK4 simulations.



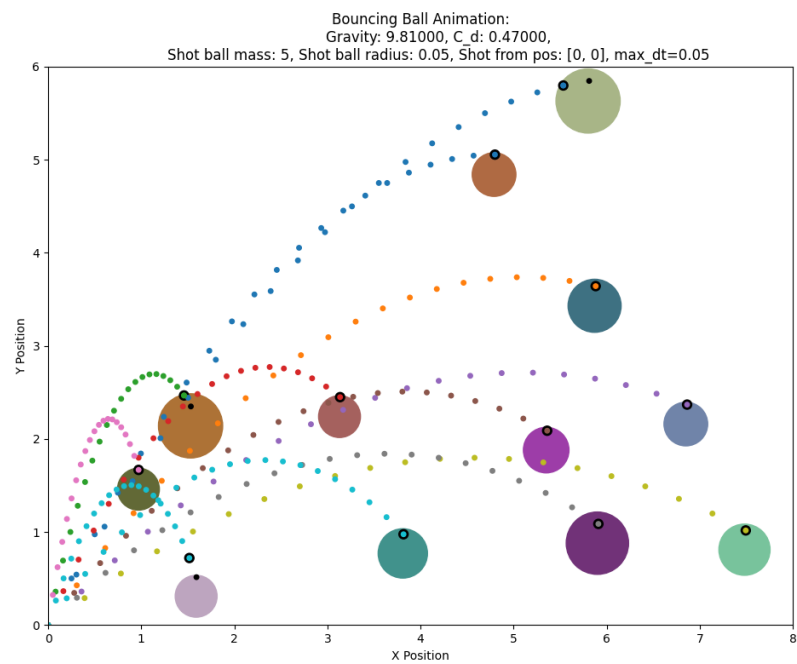
6.1.2 random_balls0 fail due to not enough frames

Simulation delta time is responsible for how smooth the animation is and defining the maximum dt for numerical methods. Thus too little of it results in unaccuracies.



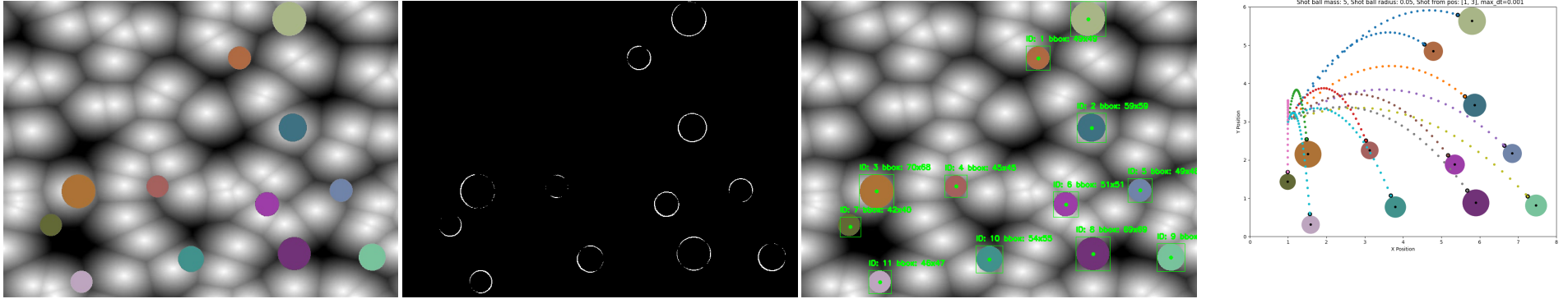
6.1.3 random_balls0 low maximum time step size

Here we observe the just enough amount of maximum dt for algorithm to work as inteded with noticable error.



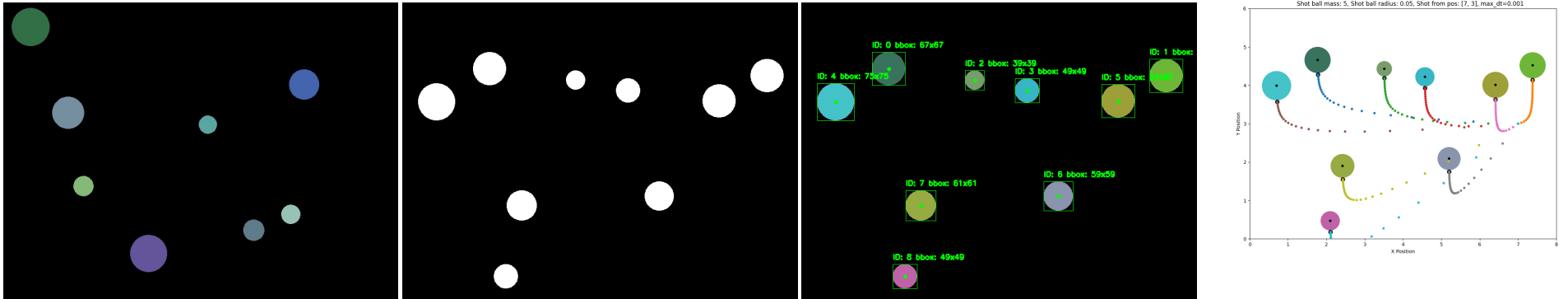
6.1.4 background_test

This is test for some complex backgrounds. The *skip_preprocessing* must be disabled and *cluster_leak* and *cluster_merge* must be used accordingly.



6.1.5 random.balls1 to the extreme

The best for the last! Here we test the extremes of the algorithm with significantly large drag and changed gravity. Also we are shooting from new place.



Chapter 7

The end

7.1 Conclusion

Task was implemented successfully with option to shoot from wherever you want and with any drag, gravity or mass value. Also sequential mode was implemented in case if you want to look at balls hit one by one rather than all at once.



